7-4 · MinIO·Qdrant 연동 · 정리·회고 0.5H

정리·회고

창고와 카드함 — 오늘 만든 파이프라인을 한 장으로

이 시간의 목표

30분 뒤에는

① MinIO 코드 여덟 줄(접속·버킷·업로드·목록·읽기…)을 한 표로 정리한다

② 멱등 적재와 「지우고 다시 넣기」 재적재를 한 문장으로 말한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ 7-5 준비(컨테이너 둘 Up · rank-bm25 설치)를 마친다

qdrant · rag-minio 가 Up 이고 lecture_kb 에 10개 포인트가 있으면 7-5 준비 완료

오늘의 요약

오늘 만든 것 — 문서를 넣어 두면 알아서 검색 가능해진다

MinIO · 창고	버킷 순회	자르기 · 임베딩	Qdrant · 카드함	역참조
raw-docs 버킷에 원본 문서	list_objects get_object	split_text embed_documents	lecture_kb 에 조각 + 벡터 + 출처	payload.source 로 원본을 꺼낸다

원본은 창고에, 색인은 카드함에 — 색인은 파생물이라 언제든 다시 만든다

멱등: uuid5(source#i) · 문서가 줄면 지우고 다시 넣기 · 원본을 되돌리면 컬렉션째 재색인

오늘의 요약

하려는 일과 코드 — 여덟 줄

하려는 일	코드
접속	Minio(endpoint, access_key=, secret_key=, secure=False)
버킷 확인·생성	mc.bucket_exists(b) / mc.make_bucket(b)
파일 올리기	mc.fput_object(bucket, key, local_path)
목록	mc.list_objects(bucket, prefix=, recursive=True)
읽기	mc.get_object(bucket, key) → read() → close()·release_conn()
고정 id	uuid.uuid5(uuid.NAMESPACE_URL, f"{source}#{i}")
조건 삭제	client.delete(name, points_selector=FilterSelector(filter=...))
조건 개수	client.count(name, count_filter=Filter(...))

get_object 뒤의 close()·release_conn() 은 finally 에 — 오류가 나도 연결을 돌려준다

오늘의 체크리스트

열다섯 항목을 여덟 줄로 — 스스로 확인해 보세요

1. 원본과 색인을 나누는 이유 · 「색인은 파생물」이 가능하게 하는 것
2. rag-minio 를 볼륨과 함께 띄우고 웹 콘솔(9001)에 로그인했다
3. 버킷 생성·업로드·목록 조회 · prefix 로 범위를 좁혔다
4. secure=True 의 SSL 오류 · 없는 키의 NoSuchKey 를 직접 봤다
5. get_object 뒤 close()·release_conn() 을 finally 에 두는 이유
6. 순회 → 자르기 → 임베딩 → 업서트가 한 번에 돌고 assert 로 멱등 확인
7. uuid4 로 중복 재현 · source 로 원본 문서 역참조
8. 줄어든 문서의 옛 조각 → 재적재로 막고, 원본을 되돌려 전체 재색인

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
SSL 오류 (wrong version number)	로컬 MinIO 는 HTTP 인데 기본값(HTTPS)로 접속	Minio(..., secure=False) 운영 HTTPS 는 True
InvalidAccessKeyId SignatureDoesNotMatch	키가 컨테이너의 ROOT_USER/PASSWORD 와 다름	docker run 에 -e 2개 명시 코드의 키와 문자까지 대조
get_object 에서 NoSuchKey	키 오타 · 한글 키의 NFC/NFD 인코딩 차이	list_objects 로 실제 키 복사 객체 키는 영문으로
반복 적재 중 연결 고갈 응답이 멈춤	get_object 응답을 close()·release_conn() 안 함	try/finally 로 감싸 반드시 반환

네 가지 모두 「한 줄」 문제 — 오류 이름(SSLError · NoSuchKey ...)을 먼저 읽는 습관

예상 질문

자주 나오는 질문 네 가지

1. **원본을 벡터DB payload 에 통째로 넣으면 안 되나요?**
   작은 규모면 된다 — 문서가 커지면 저장·백업·권한이 벡터DB 쪽 문제가 되어 버린다

2. **문서가 수천 개면 재색인이 오래 걸리지 않나요?**
   걸린다 — 그래서 접두사·수정 시각으로 바뀐 것만 골라 넣는 증분 색인을 쓴다

3. **삭제된 문서의 조각은 어떻게 지우나요?**
   버킷의 키 목록과 컬렉션의 source 목록을 비교해 없는 source 를 필터 삭제

4. **S3 를 쓰면 코드가 달라지나요?**
   엔드포인트와 키만 바뀐다 — MinIO 가 S3 호환이라 그대로 통한다

다음 차시를 위한 준비

지금 확인하고 갑니다 — 컨테이너 둘, 패키지 하나

1. **docker ps --filter name=qdrant --filter name=rag-minio …**
   rag-minio Up … 와 qdrant Up … 두 줄(실측)

2. **lecture_kb 포인트 10개**
   7단계 재색인이 끝난 상태 — 7-5 의 지식베이스

3. **pip install rank-bm25**
   Successfully installed rank-bm25-0.2.2(실측) — 7-5 키워드 검색(BM25)

준비 완료 기준

컨테이너 둘 Up

+ 포인트 10

다음 회차 예고

7-5 · Hybrid Search 구현

01
이론 1H
키워드 vs 벡터
Hybrid 의 원리

02
실습 1H
BM25 · 벡터 검색
각각 구현

03
실습 1.5H
Hybrid 결합과
주차 산출물 완성

04
정리 0.5H
정리·회고
산출물 마감

준비: 7-4 환경 그대로(qdrant · rag-minio · lecture_kb 10개) · rank-bm25 · 주차 산출물 두 가지는 7-5 에서 마감

7-4 · 정리·회고 0.5H 마무리

다음 회차는 7-5 Hybrid Search 구현

BM25 키워드 검색과 벡터 검색을 RRF 로 결합해 RAG 챗봇을 완성하고, 주차 산출물을 마감합니다

- ✓ 키워드 검색 vs 벡터 검색 — 강한 질문 유형
- ✓ rank_bm25 · Qdrant 로 각각 구현
- ✓ RRF 로 두 순위를 결합하는 함수
- ✓ RAG 답변 체인 완성 · 산출물 제출

준비할 것

컨테이너 둘 Up · rank-bm25 설치